

# Data Processing Write-Up

Research Project MCS-100

July 23, 2025

## 1 Introduction

This report details the comprehensive data processing pipeline established for a Vehicle Routing Problem (VRP) project, leading to the preparation of input data for both classical and quantum optimization algorithms. It outlines the sequential steps taken to transform raw data into structured graph representations, highlights the challenges encountered during environment setup, and summarizes the current state of the project's outputs and future directions.

## 2 Work Done (Step-by-Step Data Processing)

The project's data processing is organized into a hierarchical folder structure, reflecting a clear progression from raw inputs to highly structured graph data suitable for VRP solvers. The primary goal is to derive distance matrices for clusters of locations (waste facilities, litter bins, collection areas).

**Stage 1: 0\_raw data:** This initial stage contains the raw, unprocessed CSV datasets. These files serve as the foundational input for the entire pipeline.

- `clean.py`: Script likely used for initial data cleaning operations.
- `public-litter-bins.csv`: Raw data pertaining to public litter bin locations.
- `road-responsibility.csv`: Raw data related to road responsibility, potentially for routing constraints.
- `waste-collection-area.csv`: Raw data for waste collection areas.
- `waste-facility-locations.csv`: Raw data detailing waste facility locations.

**Stage 2: 1\_cleaned data:** Following the initial cleaning, this stage stores the refined datasets. The cleaning process typically involves handling missing values, correcting inconsistencies, and standardizing formats.

- `isolate.py`: Script likely used for further data refinement, possibly isolating specific regions or types of data.
- Cleaned versions of the raw CSVs (e.g., `public-litter-bins-clean.csv`).

**Stage 3: 2\_isolated data:** This stage contains data that has undergone isolation or spatial filtering, focusing on relevant geographical areas or subsets of data.

- `cluster.py`: Script responsible for grouping or clustering the isolated data points.
- Spatially processed versions of the cleaned CSVs (e.g., `public-litter-bins-clean-spatial.csv`).

**Stage 4: 3\_clustered data:** This stage holds the results of the clustering process, typically in JSON format, indicating how various locations are grouped.

- `complete_grapher.py`, `complete_grapher_legacy.py`: Scripts used to generate complete graphs (where every node is connected to every other node) and calculate distances within these clusters.

- Clustering results in JSON format (e.g., `public_litter_bins_to_waste_collection_areas_clustering.json`), likely containing node IDs and their cluster assignments.

**Stage 5: 4-completed graph data:** This final data processing stage contains the core input for VRP solvers: complete graphs with pre-calculated distance matrices for each cluster.

- `cluster_complete_graphs.json`, `cluster_complete_graphs_legacy.json`: These JSON files contain the distance matrices for various clusters, ready for VRP optimization.
- `naive_vrp.py`: A classical brute-force VRP solver that utilizes these `cluster_complete_graphs.json` files as its primary input to find optimal routes.
- `lookuper.py`, `sample_digester.py`: Auxiliary scripts for data lookup or digestion, supporting the overall data flow.

### 3 Hurdles Encountered (Environment and Data Handling)

While the core data processing pipeline was established, significant hurdles arose during the setup of the quantum computing environment and the handling of measurement data, which are crucial prerequisites for the quantum VRP phase.

- **IBM Quantum Platform Transition:** A major external factor was the sunset of the older IBM Quantum Platform (<https://quantum.ibm.com/sunset>) on July 1, 2025. This necessitated migrating to the new platform and understanding its updated authentication mechanisms.
- **IBM Quantum API Key Confusion:** Initial attempts to authenticate with IBM Quantum using API keys generated from the general IBM Cloud IAM dashboard (starting with `bx_`) resulted in `401 Client Error: Unauthorized` and `InvalidAccountError`. It was discovered that a specific `**IBM Quantum API token**` (a much longer key, often starting with `ibm_token:`) obtained from the user's profile on `quantum.ibm.com` was required for `QiskitRuntimeService`.
- **Qiskit Dependency Conflicts:** Persistent `ImportError` issues (e.g., `cannot import name 'BaseSampler' from 'qiskit.primitives'`) arose due to version mismatches between the core `qiskit` package and its ecosystem libraries like `qiskit-algorithms` and `qiskit-optimization`. Conflicts with other installed packages like `circuit-knitting-toolbox`, `qc-grader`, and `qiskit-serverless` further complicated installations. These were resolved by aggressively uninstalling all related Qiskit packages and then performing a clean reinstallation of only the necessary components.
- **Sampler Result Data Extraction:** Processing the raw measurement data from the `Sampler` primitive's `BitArray` object initially led to a `ValueError: The input array must have at least two axes...` This was fixed by correctly converting the `BitArray` object to a NumPy array using `np.asarray()` before further processing.
- **Measurement Count Formatting:** The raw measurement counts were in a decimal format (e.g., `Counter('3': 2062)`) or long binary strings. A custom function, `format_measurement_counts`, was developed to convert these into correctly padded binary string representations (e.g., `'00'`, `'01'`, `'10'`, `'11'`), improving readability and consistency.

### 4 Current Outputs

- **Classical VRP Solver Output:** The `naive_vrp.py` script successfully processes the `cluster_complete_graphs.json` and outputs detailed VRP results for a given cluster, including:
  - Optimal Path (e.g., `5 -> 12 -> 14 -> 5`)
  - Minimum, Maximum, and Average Distances (e.g., `Minimum Distance = 10.123 km`)
  - Potential and Average Distance Savings.

- **Quantum API Test Output:** The `quantum_api_test.py` script is now fully functional and provides clear confirmation of a working quantum environment:
  - Successful API key and instance saving.
  - Successful loading of IBM Quantum credentials and selection of a backend (e.g., `ibm_sherbrooke`).
  - Successful creation, transpilation, and execution of a simple quantum circuit (Bell state).
  - Cleanly formatted measurement counts in binary string format (e.g., `Measurement counts (bitstrings): '00': 115, '11': 135, '10': 2, '01': 4`), demonstrating correct data processing from the quantum backend.

## 5 Next Steps

With the data processing pipeline established and the quantum environment successfully configured, the next phases will focus on implementing and evaluating the quantum VRP solution.

### Next Step 1: Implement Quantum VRP (QAOA with Node-Visit-Time Encoding):

- Formulate the Vehicle Routing Problem as a Quadratic Unconstrained Binary Optimization (QUBO) problem, specifically utilizing the "Node-Visit-Time" encoding strategy as outlined in the project's methodology. This involves defining the objective function (total distance) and all VRP constraints (e.g., each customer visited exactly once, each position occupied once) as quadratic penalty terms within the QUBO.
- Integrate the formulated QUBO into Qiskit's QAOA algorithm within the `quantum_vrp.py` script.
- Develop robust decoding logic to translate the bitstring results obtained from QAOA into valid VRP routes and accurately calculate their total distances based on the provided distance matrices.

### Next Step 2: Execute on Quantum Hardware:

- Run the QAOA-based VRP solver on available IBM Quantum Processing Units (QPUs) (e.g., `ibm_sherbrooke`) to assess its performance on real quantum hardware, accounting for noise and coherence limitations.

### Next Step 3: Performance Analysis and Comparison:

- Conduct a comprehensive analysis comparing the solution quality (optimality, validity of routes) and computational efficiency (runtime) of the quantum QAOA approach against the established classical brute-force solver (`naive_vrp.py`) for various VRP instance sizes.
- Investigate the impact of QAOA-specific parameters (e.g., number of QAOA layers, choice of classical optimizer) on the quality of the quantum solution.